TASK 3 — Data Generation and ML Model for PID Self-Tuning (FINAL)

HACKSIMUL8 2026 · Department of Mechanical Engineering, PES University · 2 September 2026

> **Task 3 as stated:** *"Generate the data from the simulation experiments of the altitude control system and train a suitable ML model for self-tuning of PID gains."*

Status: COMPLETE — but PROVISIONAL. 150/150 usable samples; four model classes trained and compared on held-out data.

> ## WARNING: SUBJECT TO CHANGE BASED ON TASK 4
>
> **Everything in this document may be revised once Task 4 is finalised.** Task 3 produces a *trained model*; Task 4 is what *uses* it, and only Task 4 reveals whether that model is fit for purpose in closed loop.
>
> This has already happened once. The first Task 4 run showed the trained model extrapolating to **negative Kp** on unseen conditions, degrading performance to **168% worse than the fixed PID**. That finding forced a change in how the model is applied (adapting only Ki - see section 6), and could yet force changes to the **feature set, the cost function, the sampled envelope, or the model class**.
>
> **Specifically provisional:** the selected model class (section 5), the reported R^2 values (section 5), the choice of which gains to adapt (section 6), and the cost-function weights in `alt_cost.m`.
>
> **Not provisional:** the dataset design (section 3), the physical insight that hover thrust reveals mass (section 2), the measured Ki-versus-payload relationship (section 4), and the defects documented in section 7. Those are measurements and hold regardless of what Task 4 concludes.

---

1. Why this task exists

The problem statement makes the argument itself:

> *"highly non-linear systems subjected to system parameters' variations and environmental disturbances mitigate the performance of PID controller. For such nonlinear systems conventional off-line PID tuning techniques are not very effective. Online tuning of PID parameter is required."*

Task 2 produced a controller tuned for **one** operating condition. Task 3 exists to show that this fails when conditions change, and that the tuning can be learned instead.

The failure, measured

Task 2's gains (`Kp=30.1847, Ki=0, Kd=10.3994`) flown with an *unknown* payload:

Payload	Steady-state error
×1.0 (nominal)	0.000 m
×1.4	0.120 m
×1.8	0.239 m
×2.2	**0.359 m**

At ×2.2 the quadcopter silently settles **36 cm below** its commanded altitude, with no recovery. *(Figure: `figures/05_fixed_gains_degrade.png`)*

Why this happens connects directly to Task 2: feedback linearisation commands `U1 = m(g+u_z)/(cos ϕ cos θ)`, using the mass the controller *believes* it has. Add an unmeasured payload and that feed-forward is mis-scaled, leaving a constant deficit. With `Ki = 0` — which was optimal for the nominal case — nothing removes it.

---

2. The design decision that defines the approach

There are two ways to build "ML self-tuning PID", and the choice matters more than any hyperparameter.

(a) Gain scheduling — the obvious approach

```
features = [payload mass, wind speed]  →  model  →  [Kp, Ki, Kd]
```

The fatal question: in flight, how does the drone know its payload mass? It does not. Someone must measure it and enter it. This is a lookup table with an ML wrapper — it satisfies the wording of the task while side-stepping the engineering problem.

(b) Response-signature inference — what we built

```
fly 3 s on baseline gains → measure the response → model → [Kp, Ki, Kd]
```

The controller is **never told** the mass or the wind. It infers conditions from its own behaviour. The model learns the inverse map:

```
observed closed-loop response  →  the gains that condition requires
```

The physical insight that makes it work

Feature 6 is **mean thrust ÷ nominal weight**.

In steady hover thrust must exactly equal weight, so that ratio **is** the mass ratio. The controller reads its payload off its own thrust command — no extra sensor.

Measured across 150 conditions: thrust ratio correlates **r = +0.840** with the optimal Ki; true mass ratio correlates **r = +0.785**. The signal is real and strong.

---

3. Dataset design

Operating envelope — 150 sampled conditions

Variable	Range	Represents
mass ratio	1.0 – 2.4	payload pickup
steady wind	−1.0 – +1.0 N	sustained up/downdraught
gust	0 – 1.5 N	sudden disturbance at t = 3 s
reference step	0.5 – 2.0 m	different manoeuvre sizes
sustained tilt	0 – 0.35 rad	manoeuvring while holding altitude

Actuator limits stay at **nominal** values, so a heavy quadcopter genuinely has less thrust margin — that realism matters.

Two runs per condition

1. **Identification run** — fly 5 s with the *baseline* Task 2 gains; extract the 8-feature signature.
2. **Optimisation run** — `fminsearch` finds the gains minimising `alt_cost` for that condition. These become the training label.

The 8 features — every one measurable in flight

#	Feature	Reveals
1	overshoot	effective damping
2	rise time	effective bandwidth
3	settling time	damping and lag together
4	normalised steady-state error	disturbance magnitude, integral shortfall
5	normalised IAE	overall tracking quality
6	**mean thrust / nominal weight**	**the payload mass ratio**
7	peak thrust / weight	remaining thrust margin
8	commanded step size	manoeuvre context

None requires knowing the payload. That is the whole point.

Measured correlations (150 samples)

Pair	r
Ki vs steady-state error	**+0.863**
Ki vs thrust ratio	**+0.840**
Ki vs true mass ratio	+0.785
Kp vs overshoot	+0.701
Kd vs steady-state error	−0.482

---

4. The headline result — Ki against payload

Optimal Ki, holding other conditions fixed:

Payload	Optimal Ki	Final altitude after tuning
×1.0	**0.00**	1.0171
×1.4	5.50	**1.0000**
×1.8	11.01	0.9999
×2.2	**16.05**	**1.0000**

Ki is exactly zero at nominal and rises almost linearly with payload excess. Steady-state error goes from 36 cm to zero. ITAE improves up to 31×.

Why this is the strongest result in the project

It was predicted from theory before the data existed. The reasoning, from Task 2:

> Feedback linearisation cancels gravity, so with a *known* mass there is no steady-state error and integral action is unnecessary — which is why Task 2's optimum had Ki = 0. Add an *unknown* payload and the feed-forward is mis-scaled, leaving a constant offset that **only** integral action can remove.

The optimiser then found exactly this, independently, across 150 conditions. A prediction from physics confirmed by data is far more convincing than a fitted curve.

---

5. Model selection — four classes compared

> **PROVISIONAL - subject to change based on Task 4.** The selected class and its R^2 values depend on the current label set. If Task 4 forces a change to the cost function or the sampled envelope, the labels change and this comparison must be re-run via `task3b_retrain_compare.m`.

Choosing an architecture by assumption is not a method. Four candidates were trained on an identical 70/15/15 split and selected on **held-out** performance:

Model	R^2 Kp	R^2 Ki	R^2 Kd	mean R^2
linear least squares	0.211	0.782	0.221	0.405
quadratic + interactions	0.521	0.870	0.374	**0.589**
NN-6, Bayesian regularisation	0.454	0.822	0.144	0.473
NN-[12 8], Levenberg–Marquardt	−0.457	0.756	−0.427	−0.043

(Values from the relabelled dataset. The final run against the corrected Task 2 baseline selected NN-6 trainbr; exact figures are printed by `task3b_retrain_compare.m` *and stored in* `task3_model.mat`*.)*

Two findings worth presenting:

1. **The deep-ish network was the worst performer**, scoring below a straight line. With 150 samples and ~200 parameters it overfits. Had we assumed "ML = neural network" and shipped it, we would have presented a model worse than linear regression.
2. **Model selection on held-out data caught this.** That is the method working as intended.

---

6. Why Ki predicts well and Kp/Kd do not — an important, honest finding

> **PROVISIONAL - this section was written *because of* a Task 4 finding**, and may be revised further as Task 4 is completed.

Note the consistent pattern: **Ki R^2 ≈ 0.70–0.87, but Kp and Kd ≈ 0.2–0.5.**

That split is not random.

- **Ki is uniquely determined.** To cancel a steady-state error of a given size, integral action must have a specific strength. There is exactly one right answer, so it is learnable.
- **Kp and Kd are not.** Many (Kp, Kd) combinations produce nearly identical cost — the objective has a **flat valley** in those directions. `fminsearch` lands anywhere along it depending on its starting point, so **the training labels themselves are noisy.** No model can predict noise.

This is a property of the optimisation problem, not a failure of the method.

It has a direct practical consequence, discovered in Task 4: letting the model set all three gains caused catastrophic extrapolation on unseen conditions (predicted Kp went negative). Adapting only the parameter the data actually identifies is the correct engineering response — and the oracle confirms Kp and Kd barely need to move from their Task 2 values anyway.

If a judge asks *"why is your R^2 low?"*, this is the answer, and it is a better answer than a high R^2 with no explanation.

---

7. Four defects found and fixed during development

Being able to describe these is evidence of understanding rather than script-running.

(a) The controller could see the answer

Task 3 initially produced **0 usable samples from 150**. `sim_altitude` used a single mass `P.m` for **both** the plant and the controller's feedback linearisation — so "adding payload" also told the controller about it, and it compensated perfectly. Payload had no effect, feature 6 read 1.000 every time, and there was nothing to learn.

Fix: split into `m_ctrl` (nominal, what the controller assumes) and the plant's true mass. Immediately after, a ×1.8 payload produced 21% steady-state error and feature 6 read 1.736 ≈ the true 1.8.

(b) The optimiser brute-forced instead of integrating

With mass unknown, `fminsearch` found it could mask steady-state error with Kp ≈ 1978 rather than using integral action. Physically useless — such gains amplify sensor noise and sit permanently in saturation.

Fix: added gain regularisation (`2e-4·(Kp²+Kd²)`) to the cost, expressing the real engineering constraint.

(c) `fminsearch` never explored Ki

It builds its initial simplex from the starting point, and a component starting at exactly **zero** gets an almost invisible initial step. Starting from Ki = 0, it never tried integral action at all.

Fix: seed Ki at 5.0. This produced the clean Ki-versus-payload relationship in §4 — arguably the best result in the project, and it had been hidden behind a numerical artefact.

(d) Overshoot was under-weighted

The first cost used an overshoot weight of 0.05, so ITAE dominated and the optimiser bought settling speed with large overshoot.

Fix: raised to 0.30 and re-labelled the dataset, warm-started from the previous optima. Measured effect on the labels: mean overshoot 1.0% → 0.4%, max 7.1% → 4.1%.

---

8. Files — what each one does

Task 3 scripts

File	Purpose
`task3_generate_data_train_ml.m`	The Task 3 deliverable. Samples 150 operating conditions, runs the identification flight, extracts the 8-feature signature, optimises the gains for each condition, trains a network. Uses `parfor` across 4 workers. `FAST = true` at the top halves the sample count for a quicker run. Saves `task3_dataset.mat` and `task3_model.mat`.
`task3b_retrain_compare.m`	Trains four model classes on an identical split and selects the best by held-out mean R^2. Prints the comparison table. Overwrites `task3_model.mat` with the winner and a `predict_gains` function handle.
`task3c_relabel_warmstart.m`	Re-labels the dataset after a cost-function change, warm-starting each condition from its previous optimum so it converges in a fraction of the time. Reports old-vs-new label overshoot so the effect of the change is measured, not assumed.

Shared engine (also used by Task 2)

File	Purpose
`sim_altitude.m`	Nonlinear closed-loop simulator. **Key option for Task 3:** `m_ctrl` — the mass the *controller* assumes, held separate from the plant's true mass. This is what makes the payload genuinely unknown to the controller. Also supports `use_fbl`, tilt profiles, wind (`dist`), sensor noise, bumpless handover (`I0`, `t0`), and integration fidelity (`nsub`).
`perf_metrics.m`	Computes every metric used as a feature or a score.
`alt_cost.m`	The objective the labels are optimised against. Every weight documented inline with its justification.
`quad_params.m`, `quad_dynamics.m`	Plant model, from Task 1.

Outputs

File	Contents
`task3_dataset.mat`	`X` (150×8 features), `Y` (150×3 labels), `COND` (true conditions), `featNames`
`task3_model.mat`	trained model, `predict_gains` handle, `modelType`, train/val/test indices
`figures/06_task3_gains_vs_mass.png`	optimal gains against payload

How to run

```
cd 'C:\Users\LENOVO\Downloads\HACKSIMU8\solution'
task3_generate_data_train_ml    % ~13 min with parfor (FAST = true)
task3b_retrain_compare          % ~1 min - model class comparison
```

> **PROVISIONAL - re-run BOTH if Task 4 changes anything upstream.**
> `task3_generate_data_train_ml` reads the baseline gains from `task2_pid.mat` and the objective from `alt_cost.m`. If either changes, the dataset and the trained model are stale and must be regenerated - otherwise Task 4 evaluates a model that was trained against a different problem.

Runtime note. Data generation is the expensive step: 150 conditions × up to 180 optimiser evaluations × one 8-second simulation each ≈ 345 million derivative evaluations. Roughly 30 minutes serial, ~13 with 4 workers. The neural network itself trains in under a second — the cost is entirely in the optimisation, not the learning.

---

9. Presenting Task 3 (about 4 minutes)

1. **"Fixed gains fail when the payload changes."** Show the 36 cm steady-state error plot.
2. **"Most self-tuning schemes are *told* the mass. In flight you don't know it."** Draw the distinction between gain scheduling and true self-tuning.
3. **"Ours infers it — in hover, thrust equals weight, so thrust ratio *is* mass ratio."** Quote r = +0.84.
4. **"Here's what the optimiser found: Ki = 0 at nominal, rising linearly with payload."** Stress that this was predicted from theory *first*.
5. **"We compared four model classes and selected on held-out data."** Show the table; note the deep network scored worse than linear.
6. **"And here's what the data can and cannot tell us."** Ki is identifiable, Kp/Kd are not — flat cost valley, noisy labels.

Questions and answers

"Why not just measure the mass with a sensor?" You could, but that's a different problem. The point is that the controller's own response already contains the information. It also generalises to disturbances you cannot instrument, such as wind.

"Why is your R^2 not higher?" Because Kp and Kd are not uniquely determined — the cost has a flat valley in those directions, so the labels are intrinsically noisy. Ki, which is uniquely determined and which drives the performance gain, predicts at $R^2 \approx 0.70$–0.87.

"Why a small model rather than a deep network?" 150 samples, 8 features, a smooth underlying map. We tested this empirically — the [12 8] network scored worse than linear regression. Model size was selected on held-out data, not assumed.

"How do you know the model isn't just memorising?" 70/15/15 split with all metrics reported on the held-out test set, and the Task 4 test cases are deliberately chosen outside the training grid.

"What would you do next?" Extend from altitude to all four channels, replace the fixed identification window with continuous online adaptation, and add a Lyapunov-based stability guarantee around the learned gains.